

Créez un guide étape par étape pour configurer un nouveau projet Godot 4 avec les bindings Rust (gdext), incluant l'initialisation de la bibliothèque, la configuration du fichier Cargo.toml, la création d'une classe Rust simple et la manière de la lier à un nœud dans l'éditeur Godot 4

Guide pas à pas : Godot 4 + Rust avec gdext

Ce guide montre comment créer un projet Godot Engine utilisant les bindings Rust modernes `gdext` (`godot-rust` pour Godot 4).

À la fin, vous aurez :

- un projet Godot 4 fonctionnel,
- une bibliothèque Rust compilée en extension GDExtension,
- une classe Rust exposée à Godot,
- un nœud Godot piloté par Rust.

1. Pré-requis

Installez :

- Godot Engine (4.2+ recommandé)
- Rust via rustup
- un compilateur C/C++
 - Windows : Visual Studio Build Tools
 - Linux : `build-essential`
 - macOS : Xcode Command Line Tools

Vérifiez :

Bash

```
rustc --version  
cargo --version
```

2. Créer le projet Godot

Dans Godot :

1. Cliquez sur **New Project**
2. Nom : `godot_rust_demo`
3. Choisissez un dossier
4. Renderer : Forward+
5. Cliquez sur **Create & Edit**

Structure initiale :

```
godot_rust_demo/  
├─ project.godot
```

3. Créer la bibliothèque Rust

Dans le dossier du projet :

Bash

```
cd godot_rust_demo  
cargo new rust --lib
```

Structure :

```
godot_rust_demo/  
├─ rust/  
|   ├─ Cargo.toml  
|   └─ src/  
|       └─ lib.rs
```

4. Configurer Cargo.toml

Ouvrez :

```
rust/Cargo.toml
```

Remplacez le contenu par :

TOML

```
[package]
name = "godot_rust_demo"
version = "0.1.0"
edition = "2021"

[lib]
crate-type = ["cdylib"]

[dependencies]
godot = "0.2"

[profile.dev]
opt-level = 0

[profile.release]
opt-level = 3
```

5. Créer la bibliothèque GDExtension

Créez les dossiers suivants :

```
godot_rust_demo/
├─ rust/
├─ godot/
│   └─ rust/
```

Depuis le dossier `rust/` , compilez :

Linux

Bash

```
cargo build
```

Windows

Bash

```
cargo build
```

macOS

Bash

```
cargo build
```

La bibliothèque générée sera dans :

```
target/debug/
```

Exemples :

- Linux : `libgodot_rust_demo.so`
- Windows : `godot_rust_demo.dll`
- macOS : `libgodot_rust_demo.dylib`

6. Créer le fichier `.gdextension`

Dans le projet Godot :

```
godot_rust_demo/  
├─ rust.gdextension
```

Contenu :

INI

```
[configuration]
```

```
entry_symbol = "gdext_rust_init"
```

```
compatibility_minimum = "4.2"
```

```
[libraries]
```

```
linux.debug.x86_64 = "res://rust/target/debug/libgodot_rust_demo.so"
```

```
windows.debug.x86_64 = "res://rust/target/debug/godot_rust_demo.dll"
```

```
macos.debug = "res://rust/target/debug/libgodot_rust_demo.dylib"
```

7. Créer une classe Rust simple

Ouvrez :

```
rust/src/lib.rs
```

Remplacez par :

```
Rust
```

```

use godot::prelude::*;

struct MyExtension;

#[gdextension]
unsafe impl ExtensionLibrary for MyExtension {}

#[derive(GodotClass)]
#[class(base=Node)]
struct HelloNode {
    #[base]
    base: Base<Node>,
}

#[godot_api]
impl INode for HelloNode {
    fn init(base: Base<Node>) -> Self {
        godot_print!("HelloNode initialized!");

        Self { base }
    }

    fn ready(&mut self) {
        godot_print!("Hello from Rust!");
    }
}

```

8. Compiler le projet Rust

Depuis le dossier `rust/` :

```

Bash

cargo build

```

Si tout va bien, vous obtenez :

```

Finished dev [unoptimized + debuginfo]

```

9. Charger l'extension dans Godot

Retournez dans Godot.

Le moteur devrait détecter automatiquement :

```
rust.gdextension
```

Si nécessaire :

1. Fermez Godot
2. Réouvrez le projet

Vérifiez dans l'onglet :

```
Project > Project Settings > Plugins
```

10. Ajouter le nœud Rust dans une scène

Créer une scène

1. Nouvelle scène
2. Ajoutez un `Node`

Attacher la classe Rust

Dans l'inspecteur :

1. Cliquez sur **Attach Script**
2. Cherchez `HelloNode`

OU :

1. Cliquez sur **Add Child Node**
2. Recherchez `HelloNode`

Votre classe Rust apparaît maintenant comme un type Godot natif.

11. Tester

Lancez la scène.

Dans la console Godot :

```
HelloNode initialized!  
Hello from Rust!
```

12. Ajouter une propriété exportée

Exemple plus avancé :

```
Rust  
  
use godot::prelude::*;  
  
struct MyExtension;  
  
#[gdextension]  
unsafe impl ExtensionLibrary for MyExtension {}  
  
#[derive(GodotClass)]  
#[class(base=Node)]  
struct HelloNode {  
    #[base]  
    base: Base<Node>,  
  
    #[export]  
    speed: f32,  
}  
  
#[godot_api]  
impl INode for HelloNode {  
    fn init(base: Base<Node>) -> Self {  
        Self {  
            base,  
            speed: 10.0,  
        }  
    }  
  
    fn ready(&mut self) {  
        godot_print!("Speed = {}", self.speed);  
    }  
}
```

Dans Godot, `speed` apparaît automatiquement dans l'inspecteur.

13. Recompiler automatiquement (optionnel)

Installez :

Bash

```
cargo install cargo-watch
```

Puis :

Bash

```
cargo watch -x build
```

Chaque modification Rust recompilera automatiquement la bibliothèque.

14. Structure finale du projet

```
godot_rust_demo/  
├─ project.godot  
├─ rust.gdextension  
├─ rust/  
│   ├─ Cargo.toml  
│   ├─ src/  
│   │   └─ lib.rs  
│   └─ target/
```

15. Ressources utiles

- [godot-rust book](#)
- [godot-rust GitHub](#)
- [Godot documentation](#)

Conseils pratiques

- Utilisez `cargo build` en debug pendant le développement.
- Passez à `cargo build --release` pour les builds de production.

- Redémarrez Godot si une extension native ne recharge pas correctement.
- Les erreurs Rust apparaissent dans la console Godot ET dans le terminal Cargo.

